

LẬP TRÌNH WEB NÂNG CAO

Thái Thanh Tùng
FITHOU

BÀI 2: Điều hướng và Điều khiển

- ✓ Routing trong MVC
- ✓ Lớp Controller
- ✓ Action trong Controller

Điều hướng trong MVC

2. Điều hướng (Routing)

- ASP.NET giới thiệu Điều hướng (định tuyến) để loại bỏ nhu cầu ánh xạ từng URL bằng một tệp vật lý. Định hướng cho phép chúng ta xác định một mẫu URL ánh xạ đến trình xử lý yêu cầu. Trình xử lý yêu cầu này có thể là một tệp hoặc lớp. Trong ASP.NET ứng dụng Webform, trình xử lý yêu cầu là tệp .aspx và trong MVC, nó là lớp Controller và ActionMethod
- Ví dụ: `http://domain/students` có thể được ánh xạ tới `http://domain/studentsinfo.aspx` trong ASP.NET Webform và cùng một URL có thể được ánh xạ tới phương thức hành động Student Controller và Index trong MVC

Điều hướng không dành riêng cho khung MVC. Nó có thể được sử dụng với ASP.NET ứng dụng Webform hoặc ứng dụng MVC

Điều hướng trong MVC

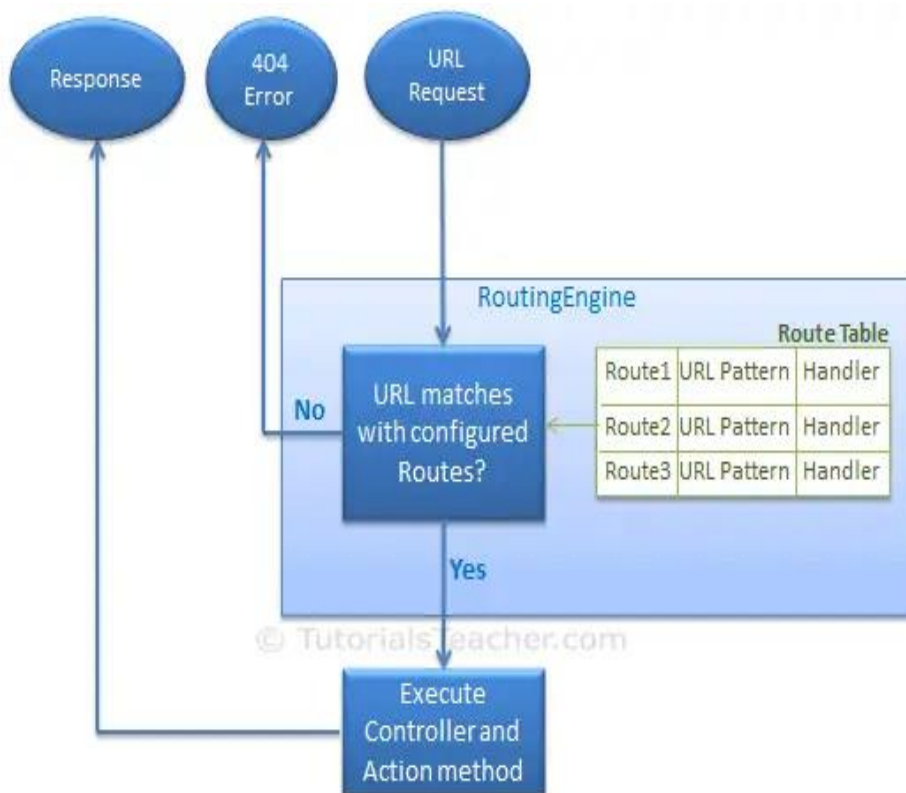
2.1. Bộ định tuyến (Route Engine)

- Routing đóng vai trò quan trọng trong MVC framework, Routing ánh xạ URL với file hoặc tệp vật lý (controller class trong MVC).
- Route chứa mẫu URL và thông tin trình xử lý, mẫu URL bắt đầu từ sau phần domain name.
- Routes có thể được cấu hình trong lớp **RouteConfig**. Có thể cấu hình nhiều routes sẽ được lưu trữ trong **RouteTable**
- Route ràng buộc việc điều hướng với các hạn chế giá trị tham số

Điều hướng trong MVC

2.1. Route Engine

Route xác định mẫu URL và thông tin trình xử lý. Tất cả các routes được cấu hình của một ứng dụng được lưu trữ trong RouteTable và sẽ được sử dụng bởi Routing engine để xác định lớp xử lý thích hợp hoặc file cho một yêu cầu đến



Hình 2.1. Routing trong MVC

Điều hướng trong MVC

2.2. Config Routing

Mỗi ứng dụng MVC được cấu hình một bộ định tuyến theo mặc định trong **Program.cs**

- Ta có thể đăng ký route với **MapControllerRoute()**
- Có thể tạo RouteConfig class để thiết lập nhiều bộ định tuyến xử lý các yêu cầu
 - name: “Default”
 - pattern: “{controller=...}/{action=...}/{id?}”

```
C#  
  
var builder = WebApplication.CreateBuilder(args);  
builder.Services.AddControllersWithViews();  
var app = builder.Build();  
if (!app.Environment.IsDevelopment())  
{  
    app.UseExceptionHandler("/Home/Error");  
    app.UseHsts();  
}  
app.UseHttpsRedirection();  
app.UseStaticFiles();  
app.UseRouting();  
app.UseAuthorization();  
app.MapControllerRoute(  
    name: "default",  
    pattern: "{controller=Home}/{action=Index}/{id?}");  
app.Run();
```

Hình 2.2. RouteConfig trong MVC

Điều hướng trong MVC

2.2. Config Routing

Mô hình URL được xem xét sau phần tên miền trong URL có dạng thức: **"{controller}/{action}/{id}"**

Ví dụ: https://localhost:1234/{controller}/{action}/{id}

- Bất cứ gì sau "localhost:1234/" sẽ được coi là tên bộ điều khiển. Tương tự, bất cứ gì sau bộ điều khiển sẽ được coi là tên hành động và sau đó là giá trị của tham số id



Hình 2.3. URLConfig mapping trong MVC

Điều hướng trong MVC

2.2. Config Routing

Nếu URL không chứa bất cứ thứ gì sau tên miền, thì bộ điều khiển và phương thức hành động mặc định sẽ xử lý yêu cầu. Ví dụ, `http://localhost:1234` sẽ được xử lý bởi **HomeController** và phương thức **Index()** như được cấu hình trong tham số mặc định. Ta có thể cấu hình điều hướng tùy chỉnh bằng phương thức mở rộng `MapControllerRoute()`, khi đó ta cần cung cấp ít nhất hai tham số trong `MapRoute`, tên định tuyến và mẫu URL.

Default Route

- Định nghĩa trong tệp `Program.cs`
- Ta có thể config lại để thay đổi trang mặc định

C#

```
app.MapControllerRoute(  
    name: "default",  
    pattern: "{controller=Home}/{action=Index}/{id?}");
```


Điều hướng trong MVC

2.3. RouteTable

- Thiết lập nhiều bộ định tuyến với MapControllerRoute()
- Routing sẽ map tuần tự theo khai báo các bộ định tuyến

MVC điều hướng theo trình tự, bắt đầu với tuyến được cấu hình đầu tiên và nếu URL đến không thỏa mãn mẫu URL của bộ định tuyến, thì nó sẽ đánh giá tuyến thứ hai, v.v.

```
app.MapControllerRoute(  
    name: "default",  
    pattern: "{controller=Home}/{action=Index}/{id?}");  
  
app.MapControllerRoute(  
    name: "Student",  
    pattern: "Student/{*Details}",  
    defaults: new { controller = "Student", action = "Details" });  
  
app.MapControllerRoute(  
    name: "StudentDefault",  
    pattern: "{controller=Student}/{action=Index}/{id?}");  
  
app.Run();
```

Điều hướng trong MVC

2.3. RouteTable

- Thiết lập thuộc tính cho các phương thức xử lý (ActionMethod)
- Cho phép điều hướng nhiều URL Config về cùng bộ định tuyến

Cú pháp: *[Route("[Controller]/[Action]")]*

```
0 references
public class StudentController : Controller
{
    [Route("")]
    [Route("Home")]
    [Route("Home/Index")]
    0 references
    public IActionResult Index()
    {
        return View();
    }
}
```

CONTROLLER TRONG ASP.NET MVC

2. Điều khiển trong MVC (Controller)

- Bộ điều khiển trong kiến trúc MVC xử lý bất kỳ yêu cầu URL đến nào. Controller là một lớp, có nguồn gốc từ lớp cơ sở `System.Web.Mvc.Controller`. Lớp controller chứa các phương thức public được gọi là Action methods. Bộ điều khiển và phương thức hành động của nó xử lý các yêu cầu trình duyệt đến, truy xuất dữ liệu mô hình cần thiết và trả về các phản hồi thích hợp.
- Trong ASP.NET MVC, mọi tên lớp controller phải kết thúc bằng một từ "Controller". Ví dụ: tên controller trang chủ phải là `HomeController` và đối với trang sinh viên, nó phải là `StudentController`. Ngoài ra, mọi lớp điều khiển phải được đặt trong thư mục Controller của cấu trúc thư mục MVC.

CONTROLLER TRONG ASP.NET MVC

2.1. Lớp Controller

- Controller xử lý các yêu cầu URL đến. Định tuyến MVC gửi yêu cầu đến bộ điều khiển và phương thức hành động thích hợp dựa trên URL và các tuyến được cấu hình.
- Tất cả các phương thức có phạm vi **public** trong class Controller được gọi là **Action methods**.
- Lớp Controller phải được dẫn xuất từ lớp `System.Web.Mvc.Controller`.
- Tên lớp Controller phải kết thúc bằng "Controller".
- Một bộ điều khiển mới có thể được tạo bằng cách sử dụng các mẫu template khác nhau, ta có thể tạo template tùy chỉnh.

CONTROLLER TRONG ASP.NET MVC

2.1. Lớp Controller

Cung cấp các phương thức phản hồi các yêu cầu HTTP được gửi đến từ trang ASP.NET MVC

Namespace: System.Web.Mvc

- Object → ControllerBase → Controller

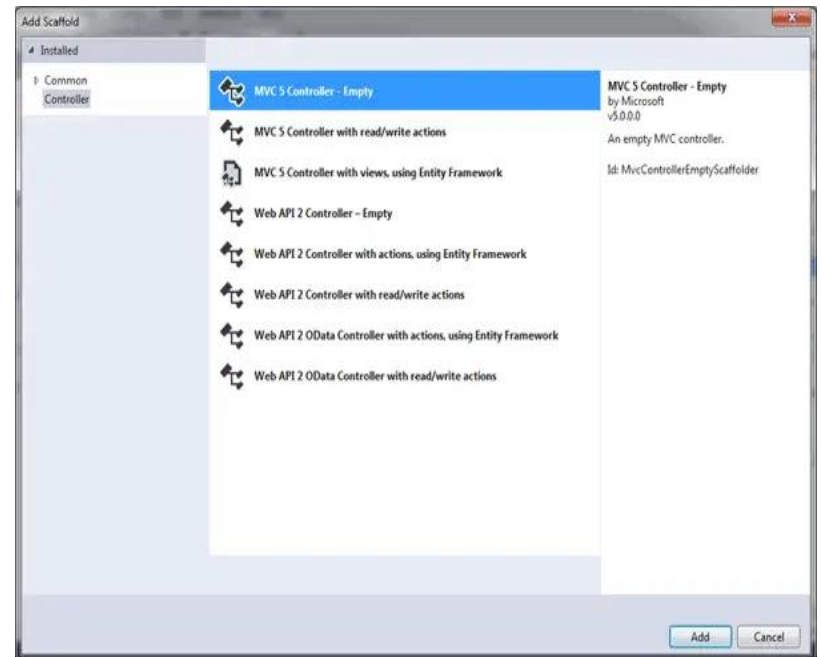
<https://learn.microsoft.com/en-us/dotnet/api/system.web.mvc.controller?view=aspnet-mvc-5.2>

ActionInvoker	Gets the action invoker for the controller.
AsyncManager	Provides asynchronous operations.
Binders	Gets or sets the binder.
ControllerContext	Gets or sets the controller context. (Inherited from ControllerBase)
DisableAsyncSupport	Gets whether to disable the asynchronous support for the controller.
HttpContext	Gets HTTP-specific information about an individual HTTP request.
ModelState	Gets the model state dictionary object that contains the state of the model and of model-binding validation.
Profile	Gets the HTTP context profile.
Request	Gets the HttpRequestBase object for the current HTTP request.
Resolver	Represents a replaceable dependency resolver providing services. By default, it uses the System.Web.Mvc.DependencyResolver.CurrentCache .
Response	Gets the HttpResponseBase object for the current HTTP response.
RouteData	Gets the route data for the current request.
Server	Gets the HttpServerUtilityBase object that provides methods that are used during Web request processing.
Session	Gets the HttpSessionStateBase object for the current HTTP request.
TempData	Gets or sets the dictionary for temporary data. (Inherited from ControllerBase)
TempDataProvider	Gets the temporary-data provider object that is used to store data for the next request.
Url	Gets the URL helper object that is used to generate URLs by using routing.
User	Gets the user security information for the current HTTP request.
ValidateRequest	Gets or sets a value that indicates whether request validation is enabled for this request. (Inherited from ControllerBase)
ValueProvider	Gets or sets the value provider for the controller. (Inherited from ControllerBase)
ViewBag	Gets the dynamic view data dictionary. (Inherited from ControllerBase)
ViewData	Gets or sets the dictionary for view data. (Inherited from ControllerBase)
ViewEngineCollection	Gets the view engine collection.

CONTROLLER TRONG ASP.NET MVC

2.2. Tạo điều khiển

Chọn template tùy chỉnh



Hình 2.5. Đăng ký Route trong Global.asax

CONTROLLER TRONG ASP.NET MVC

2.1. Tạo điều khiển

- Lớp StudentController kế thừa từ lớp Controller
- Tất cả controller trong MVC phải được bắt kế thừa từ lớp trừu tượng Controller này.
- Lớp Controller cơ sở này chứa các phương thức trợ giúp được sử dụng cho các mục đích khác nhau

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;
using System.Web.Mvc;

namespace MVC_BasicTutorials.Controllers
{
    public class StudentController : Controller
    {
        // GET: Student
        public ActionResult Index()
        {
            return View();
        }
    }
}
```

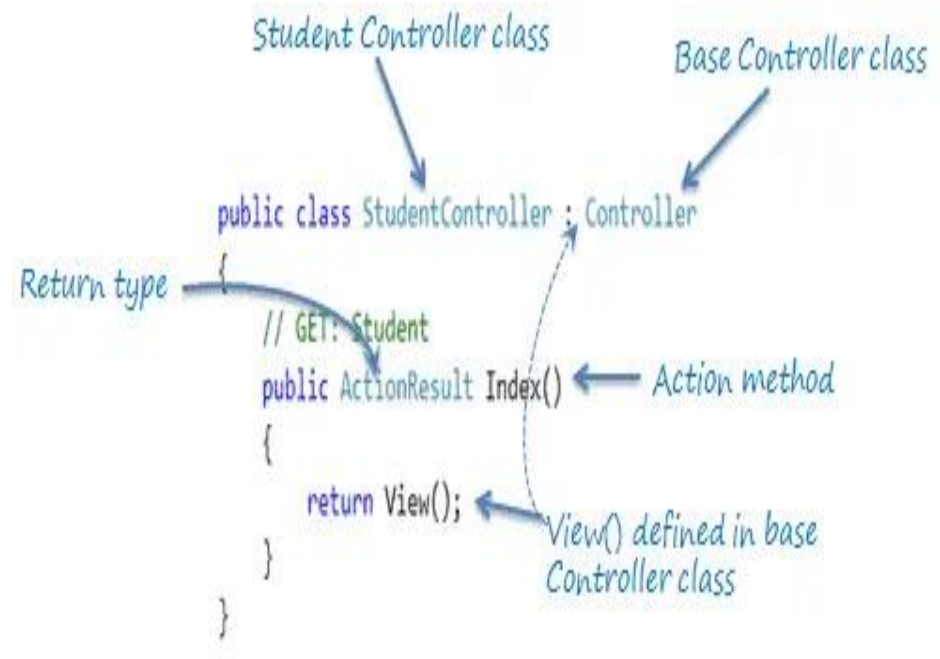
**Chú ý: Đổi kiểu ActionResult thành String
return View với kiểu dữ liệu khác nhau**

ActionMethod và ActionResult

2.2. Action Method

Tất cả các phương thức public của lớp Controller được gọi là **Action Methods**. Chúng giống như bất kỳ phương pháp thông thường nào khác với những đặc điểm hạn chế sau:

- Phải có phạm vi là **public**
- Không được phép nạp chồng hàm Action Methods
- Không sử dụng loại hàm tĩnh với từ khóa **static**



Hình 2.6. Action Method mặc định là `Index()` trong Controller

ActionMethod và ActionResult

2.2. Action Method

Default ActionMethod:

Mỗi controller có thể có một phương thức hành động mặc định theo route được cấu hình trong class RouteConfig là phương thức Index().

Tuy nhiên, ta có thể thay đổi tên hành động mặc định theo yêu cầu của trong lớp RouteConfig

```
routes.MapRoute(  
    name: "Default",  
    url: "{controller}/{action}/{id}/{name}",  
    defaults: new {controller = "Home",  
                    action = "Index",  
                    id = UrlParameter.Optional  
});
```

ActionMethod và ActionResult

2.2. Action Method

Parameters:

ActionMethod có thể chứa các tham số với kiểu dữ liệu khác nhau (nguyên thủy hoặc đối tượng)

Mặc định, tham số được trích xuất từ tập hợp data collection của đối tượng Request. Thường là tập hợp name/values của Form, QueryString, Cookies ...

ASP.NET MVC tự động map các tham số của ActionMethod nếu có cùng tên

```
[HttpPost]
public ActionResult Edit(Student std)
{
    // update student to the database

    return RedirectToAction("Index");
}

[HttpDelete]
public ActionResult Delete(int id)
{
    // delete student from the database whose id matches with
    // specified id

    return RedirectToAction("Index");
}
```

ActionMethod và ActionResult

2.3. ActionResult

- MVC bao gồm các lớp Kết quả trả về (ActionResult) khác nhau, có thể được trả về từ một ActionMethod.
- Các lớp kết quả đại diện cho các loại phản hồi khác nhau, chẳng hạn như HTML, Tập, Chuỗi, JSON, javascript, v.v.
- Lớp **ActionResult** là lớp cơ sở của tất cả các lớp kết quả trả về, vì vậy nó có thể là kiểu trả về của bất kỳ ActionMethod. Tuy nhiên, ta có thể chỉ định lớp kết quả thích hợp làm phương thức hành động kiểu trả về.

ActionMethod và ActionResult

2.3. ActionResult

- ActionResult là kiểu trả về của các ActionMethod. Tùy thuộc vào kiểu trả về ta sử dụng các phương thức tương ứng của Controller.

Result Class	Description	Base Controller Method
ViewResult	Represents HTML and markup.	View()
EmptyResult	Represents No response.	
ContentResult	Represents string literal.	Content()
FileContentResult, FilePathResult, FileStreamResult	Represents the content of a file.	File()
JavaScriptResult	Represents a JavaScript script.	JavaScript()
JsonResult	Represents JSON that can be used in AJAX.	Json()
RedirectResult	Represents a redirection to a new URL.	Redirect()
RedirectToRouteResult	Represents redirection to another route.	RedirectToRoute()
PartialViewResult	Represents the partial view result.	PartialView()
HttpUnauthorizedResult	Represents HTTP 403 response.	

Hình 2.7. ActionResult và phương thức trả dữ liệu Controller

ActionMethod và ActionResult

2.4. Action Selectors

ActionSelector là thuộc tính thiết lập cho ActionMethod để RouteEngine lựa chọn Method xử lý yêu cầu:

- ActionName
- NoAction
- ActionVerbs

ActionName được dùng để gọi thay cho ActionMethod

- <https://localhost:1234/Student/Find/100>

Chú ý: NoAction sử dụng khi ta tạo phương thức thông thường của Controller Class mà không muốn nó trở thành ActionMethod

Action Selector: ActionName

```
public class StudentController : Controller
{
    public StudentController()
    {
    }

    [ActionName("Find")]
    public ActionResult GetById(int id)
    {
        // get student from the database
        return View();
    }
}
```

ActionMethod và ActionResult

2.4. Action Selectors

ActionVerbs để xử lý các yêu cầu khác nhau của `HttpRequest`:

`HttpGet`, `HttpPost`, `HttpPut`, `HttpDelete`, `HttpOptions`, `HttpPatch`

Ta có thể thiết lập nhiều Verbs cho `ActionMethod` để xử lý yêu cầu khác nhau. Verb mặc định là `HttpGet`

Tham khảo HTTP Semantic tại: RFC 9110

<https://www.rfc-editor.org/rfc/rfc9110.html>

Http method	Usage
GET	To retrieve the information from the server. Parameters will be appended in the query string.
POST	To create a new resource.
PUT	To update an existing resource.
HEAD	Identical to GET except that server do not return the message body.
OPTIONS	It represents a request for information about the communication options supported by the web server.
DELETE	To delete an existing resource.
PATCH	To full or partial update the resource.

Hình 2.8. `ActionSelector` `ActionVerbs`

ActionMethod và ActionResult

2.4. Action Selectors

ActionVerbs để xử lý các yêu cầu khác nhau của `HttpRequest`:

`HttpGet`, `HttpPost`, `HttpPut`, `HttpDelete`, `HttpOptions`, `HttpPatch`

Cú pháp:

```
[HttpMethod]
public type ActionMethod() {
    return View();
}
```

```
public class StudentController : Controller
{
    public ActionResult Index() // handles GET requests by default
    {
        return View();
    }

    [HttpPost]
    public ActionResult PostAction() // handles POST requests by default
    {
        return View("Index");
    }

    [HttpPut]
    public ActionResult PutAction() // handles PUT requests by default
    {
        return View("Index");
    }

    [HttpDelete]
    public ActionResult DeleteAction() // handles DELETE requests by default
    {
        return View("Index");
    }

    [HttpHead]
    public ActionResult HeadAction() // handles HEAD requests by default
    {
        return View("Index");
    }

    [HttpOptions]
    public ActionResult OptionsAction() // handles OPTION requests by default
    {
        return View("Index");
    }

    [HttpPatch]
    public ActionResult PatchAction() // handles PATCH requests by default
    {
        return View("Index");
    }
}
```

ActionMethod và ActionResult

2.4. Action Selectors

AcceptVerbs để gộp nhiều lệnh verbs cho một ActionMethod

Cú pháp:

`[AcceptVerbs(HttpVerbs.Post | HttpVerbs.Get)]`

```
[AcceptVerbs(HttpVerbs.Post | HttpVerbs.Get)]  
public ActionResult GetAndPostAction()  
{  
    return RedirectToAction("Index");  
}
```


KẾT THÚC

Q & A

XIN CHÀO VÀ HẸN GẶP LẠI TUẦN SAU

Routing và Controller

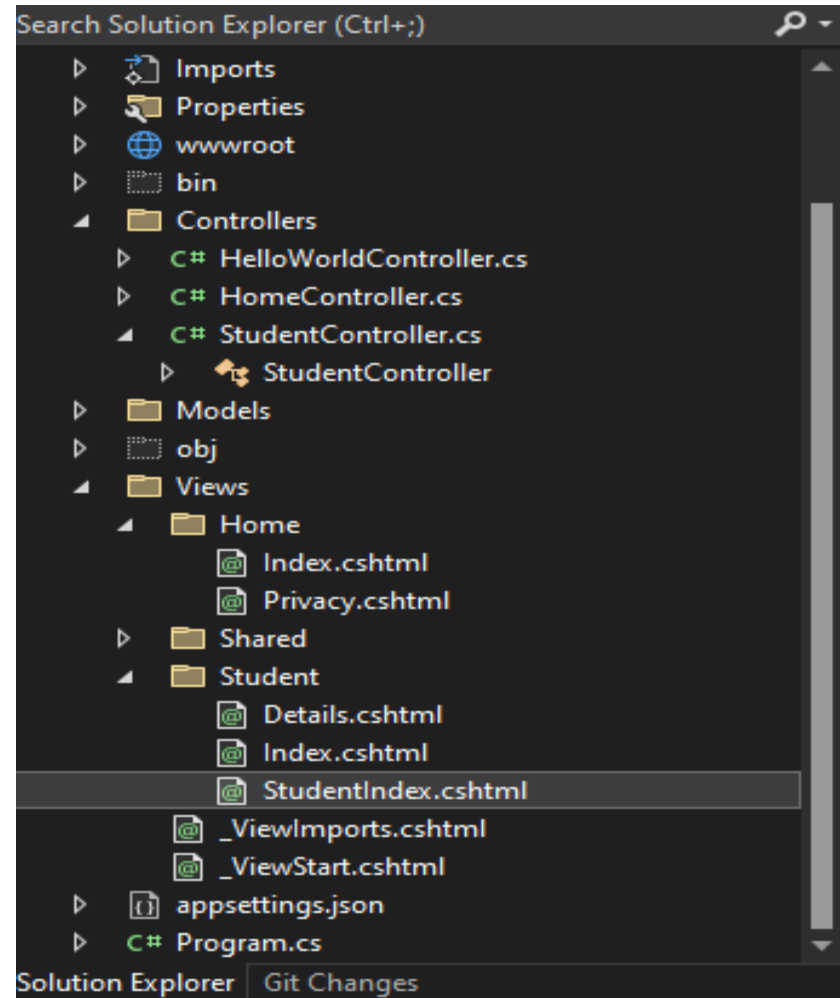
+ Program.cs

+ Controller:

- HomeController
- HelloWorldController
- StudentController

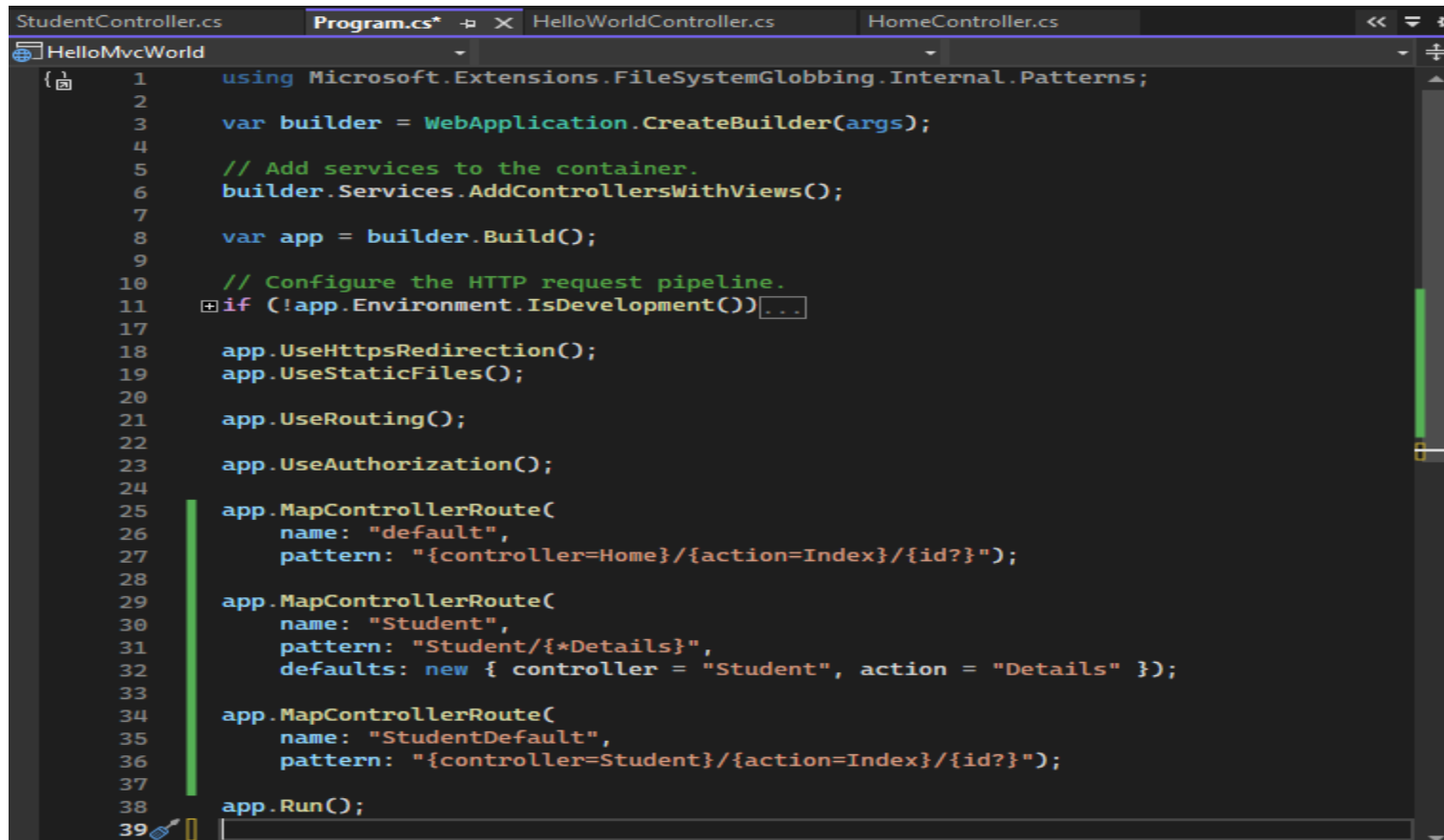
+ View:

- Home\Index.cshtml
- Student\Index.cshtml
- Student\Details.cshtml



Routing và Controller

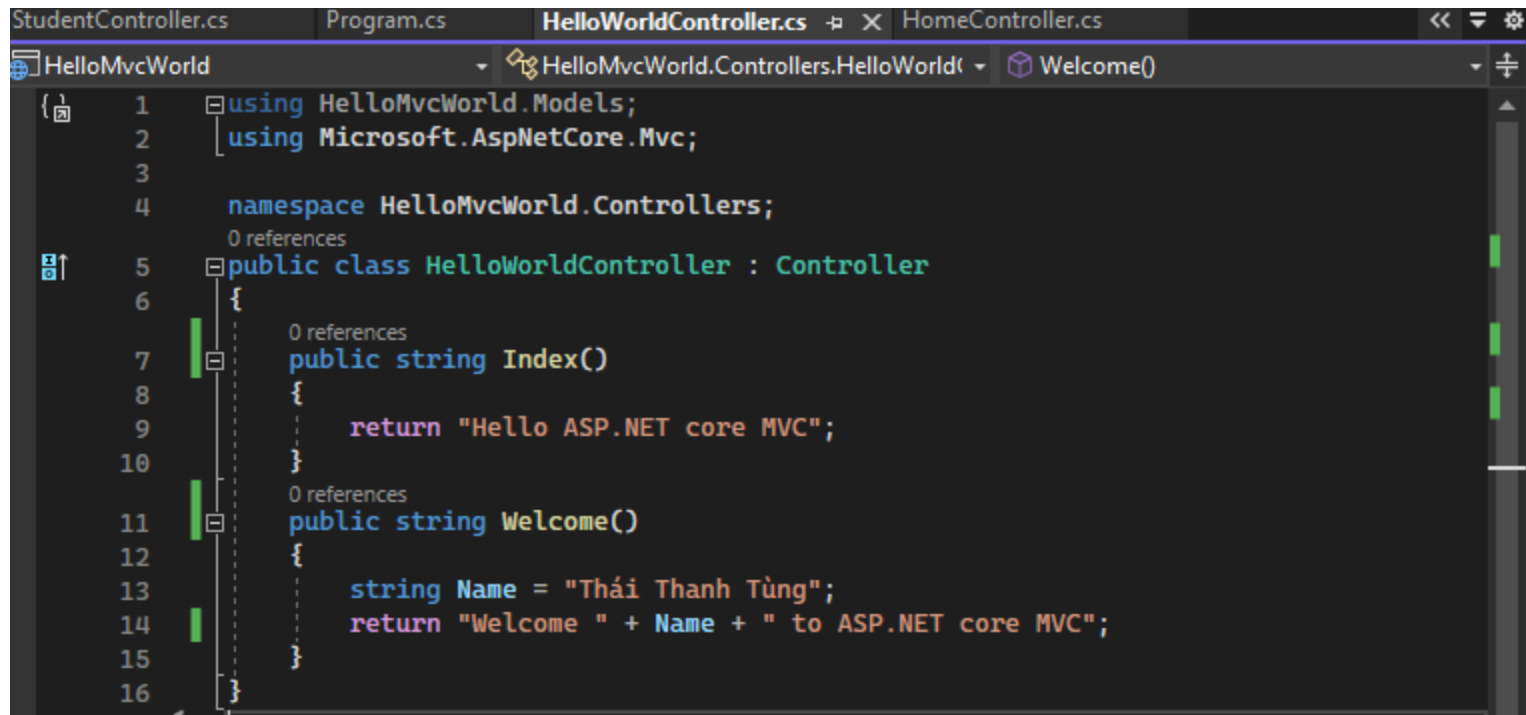
+ Program.cs: MapControllerRoute()



```
StudentController.cs  Program.cs*  HelloWorldController.cs  HomeController.cs
HelloMvcWorld
{
1  using Microsoft.Extensions.FileSystemGlobbing.Internal.Patterns;
2
3  var builder = WebApplication.CreateBuilder(args);
4
5  // Add services to the container.
6  builder.Services.AddControllersWithViews();
7
8  var app = builder.Build();
9
10 // Configure the HTTP request pipeline.
11 if (!app.Environment.IsDevelopment())
12 {
13     app.UseHttpsRedirection();
14     app.UseStaticFiles();
15 }
16
17 app.UseRouting();
18
19 app.UseAuthorization();
20
21 app.MapControllerRoute(
22     name: "default",
23     pattern: "{controller=Home}/{action=Index}/{id?}");
24
25 app.MapControllerRoute(
26     name: "Student",
27     pattern: "Student/{*Details}",
28     defaults: new { controller = "Student", action = "Details" });
29
30 app.MapControllerRoute(
31     name: "StudentDefault",
32     pattern: "{controller=Student}/{action=Index}/{id?}");
33
34 app.Run();
35 }
```

Routing và Controller

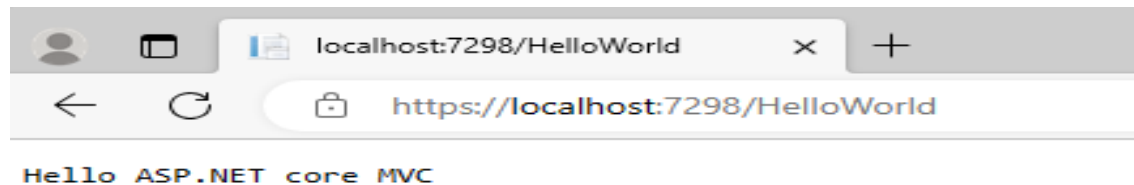
- + HomeController.cs (default)
- + HelloWorldController.cs: ActionMethod Index(), Welcome()



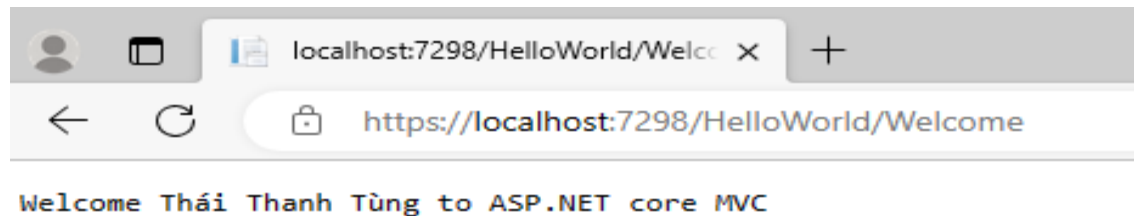
```
1 using HelloMvcWorld.Models;
2 using Microsoft.AspNetCore.Mvc;
3
4 namespace HelloMvcWorld.Controllers;
5 public class HelloWorldController : Controller
6 {
7     public string Index()
8     {
9         return "Hello ASP.NET core MVC";
10    }
11    public string Welcome()
12    {
13        string Name = "Thái Thanh Tùng";
14        return "Welcome " + Name + " to ASP.NET core MVC";
15    }
16 }
```

Routing và Controller

+ HelloWorldController - Index



+ HelloWorldController - Welcome



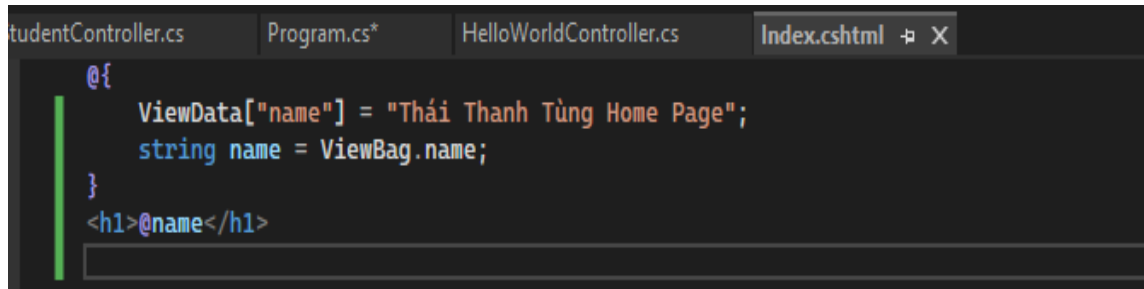
Routing và Controller

+ StudentController.cs: Thuộc tính [Route({controller/action})]

```
1 using HelloMvcWorld.Models;
2 using Microsoft.AspNetCore.Mvc;
3
4 namespace HelloMvcWorld.Controllers
5 {
6     public class StudentController : Controller
7     {
8         [Route("")]
9         [Route("Home")]
10        [Route("Home/Index")]
11        public IActionResult Index()
12        {
13            return View();
14        }
15
16        [Route("Home/Details")]
17        public ActionResult Details(int id)
18        {
19            ViewBag.Id = id;
20            return View();
21        }
22    }
23 }
24
```

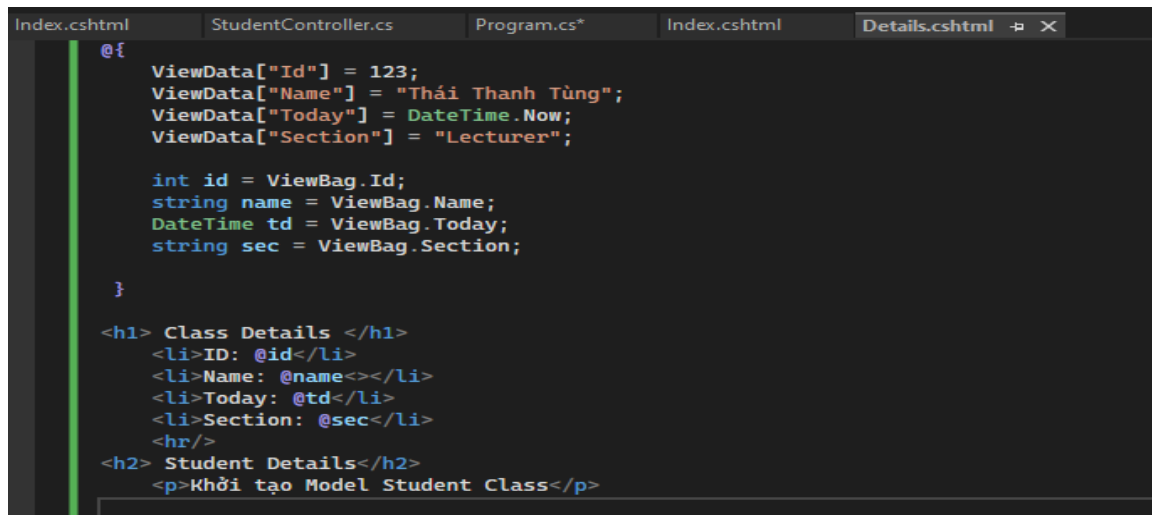
Routing và Controller

+ Student\Index.cshtml



```
@{
    ViewData["name"] = "Thái Thanh Tùng Home Page";
    string name = ViewBag.name;
}
<h1>@name</h1>
```

+ Student\Details.cshtml



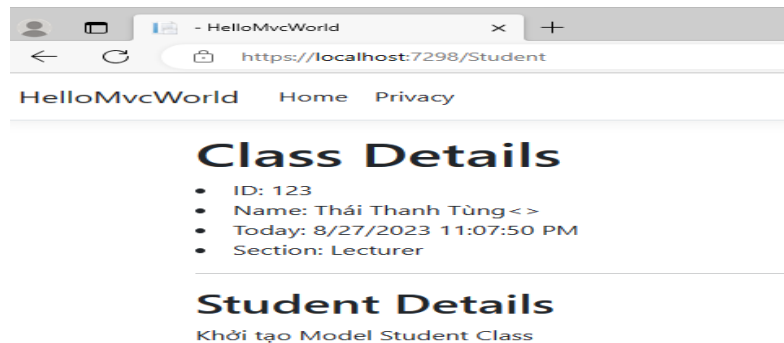
```
@{
    ViewData["Id"] = 123;
    ViewData["Name"] = "Thái Thanh Tùng";
    ViewData["Today"] = DateTime.Now;
    ViewData["Section"] = "Lecturer";

    int id = ViewBag.Id;
    string name = ViewBag.Name;
    DateTime td = ViewBag.Today;
    string sec = ViewBag.Section;
}

<h1> Class Details </h1>
<li>ID: @id</li>
<li>Name: @name</li>
<li>Today: @td</li>
<li>Section: @sec</li>
<hr/>
<h2> Student Details</h2>
<p>Khởi tạo Model Student Class</p>
```

Routing và Controller

+ Student - Index



+ Student - Details

